Compiladores e Intérpretes
Conceptos Generales

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Problema: ¿Cómo ejecutamos un programa?

Supongamos que tenemos un programa en un lenguaje de alto nivel tipo Java

Programa 1

```
if(id1>10)
    var= “hola”;
else
    print(“chau”);
//fin
```

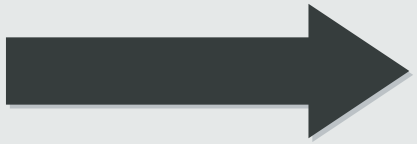


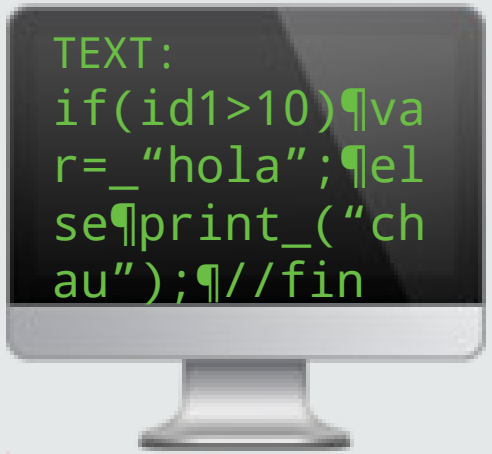
¿Cómo hacemos para **ejecutar** el programa en un dispositivo?

Problema: ¿Cómo ejecutamos un programa?

Programa 1

```
if(id1>10)
    var= “hola”;
else
    print(“chau”);
//fin
```





A simple vista para el dispositivo nuestro programa es un **texto** (secuencia de caracteres)

Proble[illegible]os un programa?

El problema es que el dispositivo no entiende como instrucciones las sentencias de nuestro programa, porque no entiende de forma directa el lenguaje en el que están escritas

Programa 1

```
if(id1>10)
    var= “hola”;
else
    print(“chau”);
//fin
```



Ejecutar!

A simple vista para el dispositivo nuestro programa es un **texto** (secuencia de caracteres)

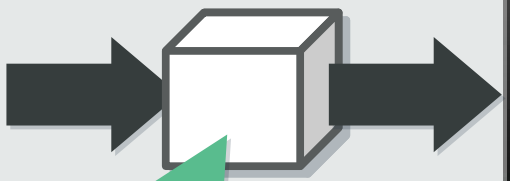
Si lo tratamos de ejecutar directamente, el dispositivo no va a hacer nada o nos va a dar un error

Proble[illegible]os un programa?

El problema es que el dispositivo no entiende como instrucciones las sentencias de nuestro programa, porque no entiende de forma directa el lenguaje en el que están escritas

Programa 1

```
if(id1>10)
    var= “hola”;
else
    print(“chau”);
//fin
```





Ejecutar!

A simple vista para el dispositivo nuestro programa es un **texto** (secuencia de caracteres)

Si lo tratamos de ejecutar directamente, el dispositivo no va a hacer nada o nos va a dar un error

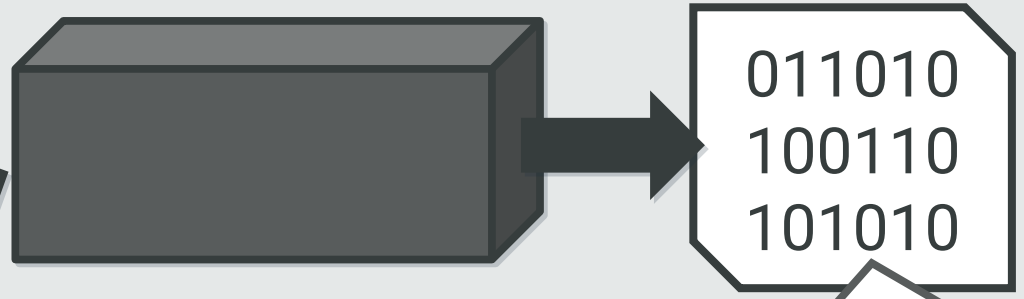
La solución es **traducir** el programa a un **programa equivalente** pero que el **dispositivo** pueda **entender**

Problema: ¿Cómo ejecutamos un programa?

- Dos principales estrategias

Programa 1

```
if(id1>10)
    var= “hola”;
else
    print(“chau”);
//fin
```

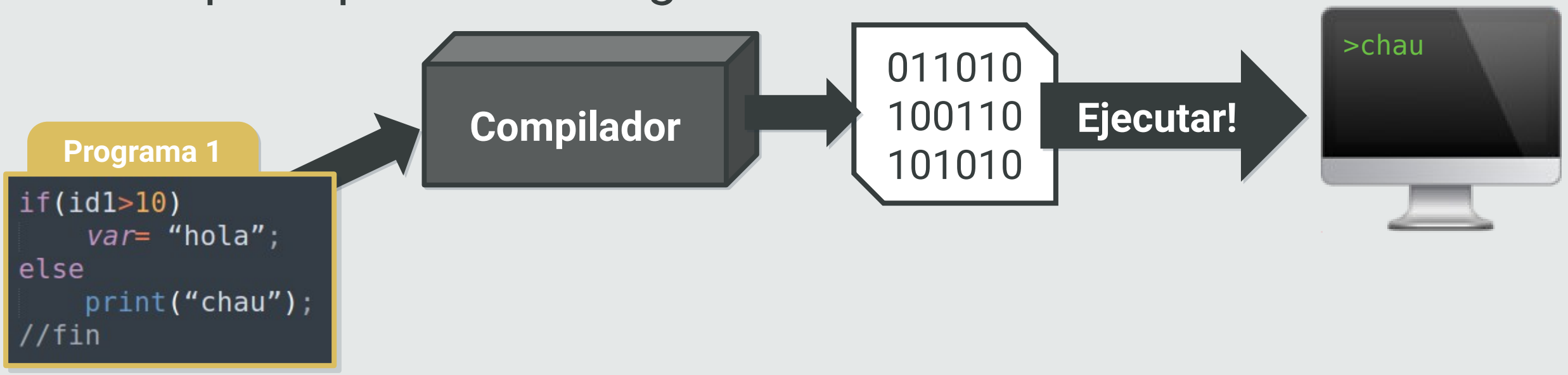




Traducción por completo Programa1 a código de una **máquina destino**

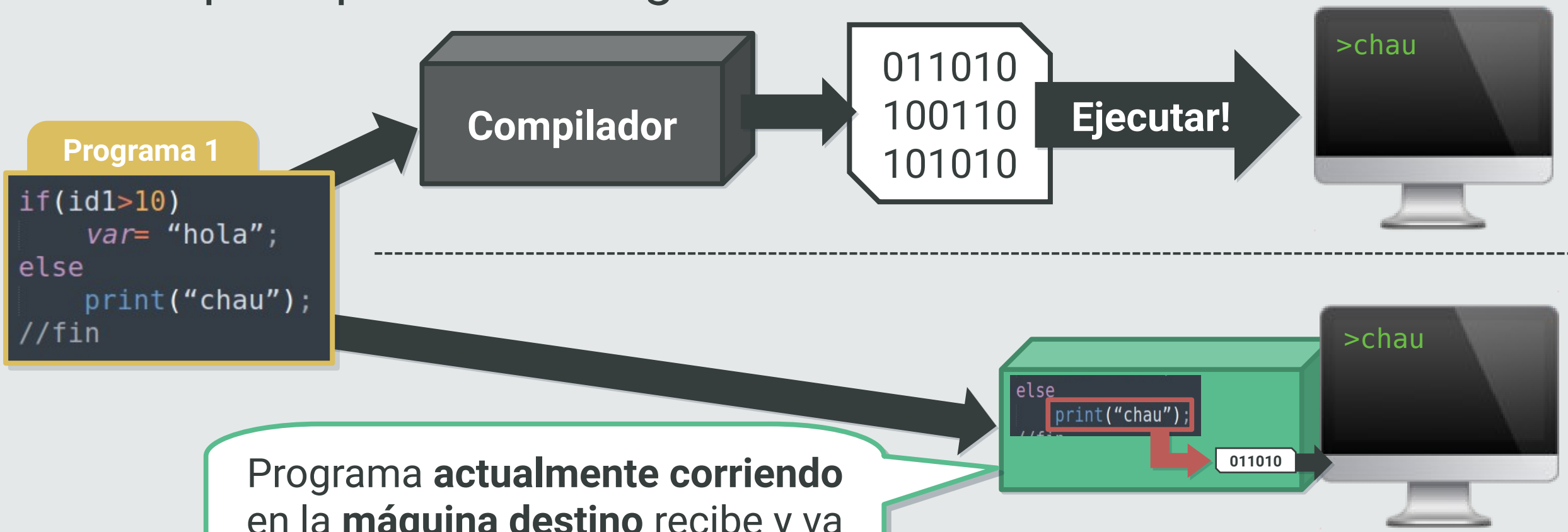
Problema: ¿Cómo ejecutamos un programa?

- Dos principales estrategias



Problema: ¿Cómo ejecutamos un programa?

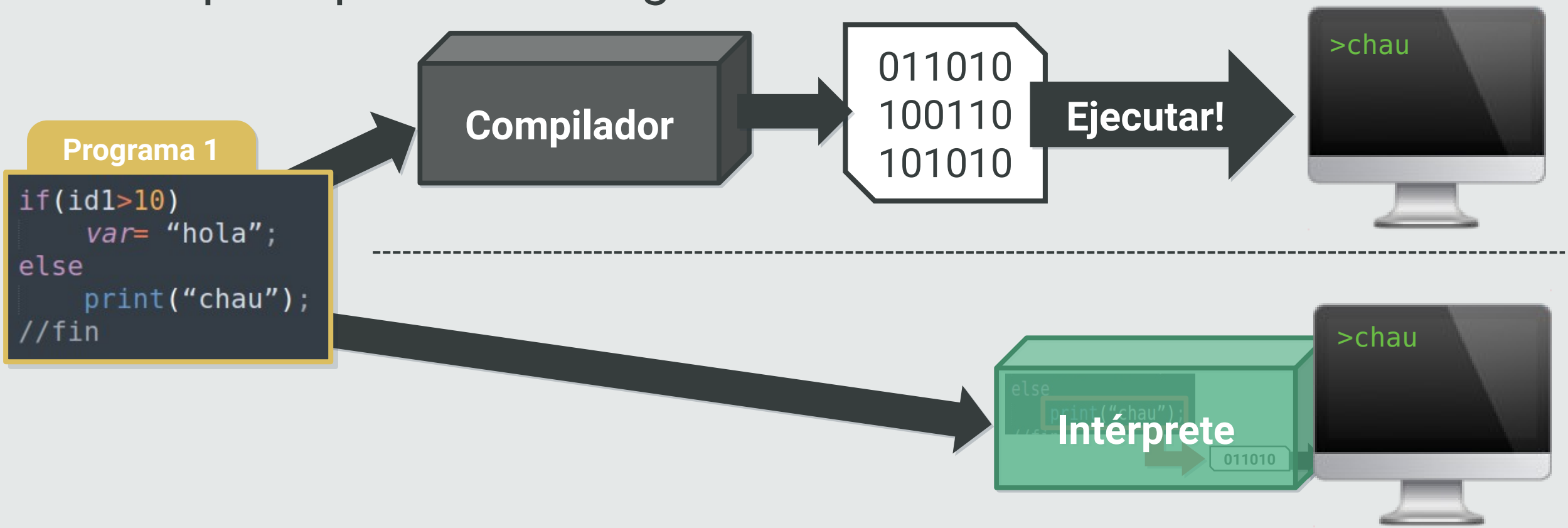
- Dos principales estrategias



Programa **actualmente corriendo** en la **máquina destino** recibe y va ejecutando Programa1

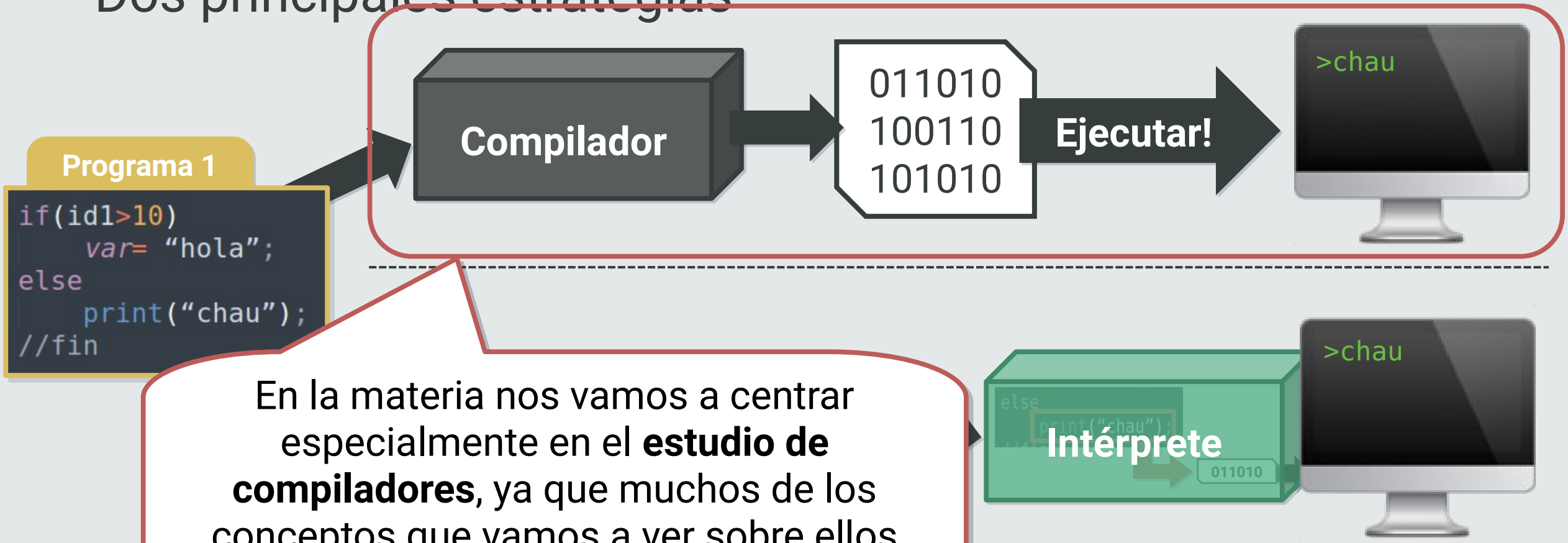
Problema: ¿Cómo ejecutamos un programa?

- Dos principales estrategias



Problema: ¿Cómo ejecutamos un programa?

- Dos principales estrategias

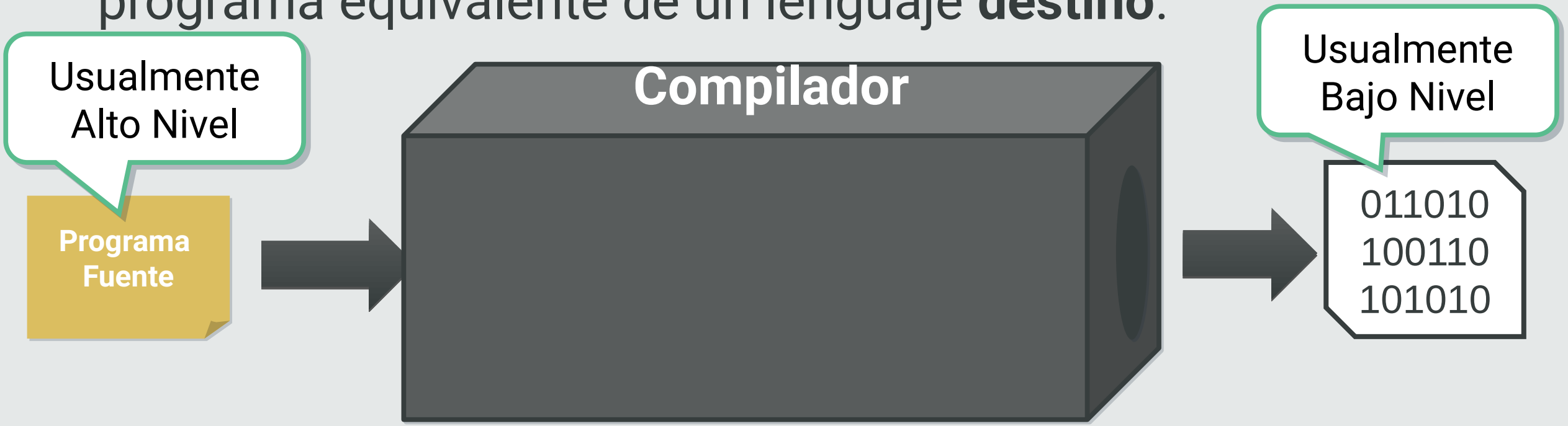


En la materia nos vamos a centrar especialmente en el **estudio de compiladores**, ya que muchos de los conceptos que vamos a ver sobre ellos también son aplicables a intérpretes

Características de un Compilador

Características de un Compilador

- Un **Compilador** es un software que **traduce** automáticamente un programa de un lenguaje **fuente** a un programa equivalente de un lenguaje **destino**.



Características de un Compilador

- Un **Compilador** es un software que **traduce** automáticamente un programa de un lenguaje **fuente** a un ivalente de un lenguaje **destin**

Lenguaje Alto Nivel

- Independiente de la máquina
- Sintaxis compleja
- Semántica compleja
- Anidamiento
- Subprogramas
- Objetos

…



Lenguaje Bajo Nivel

- Específico
- Sintaxis simple
- Semántica simple
- Manejo de memoria
- Registros
- Alcance simple

…

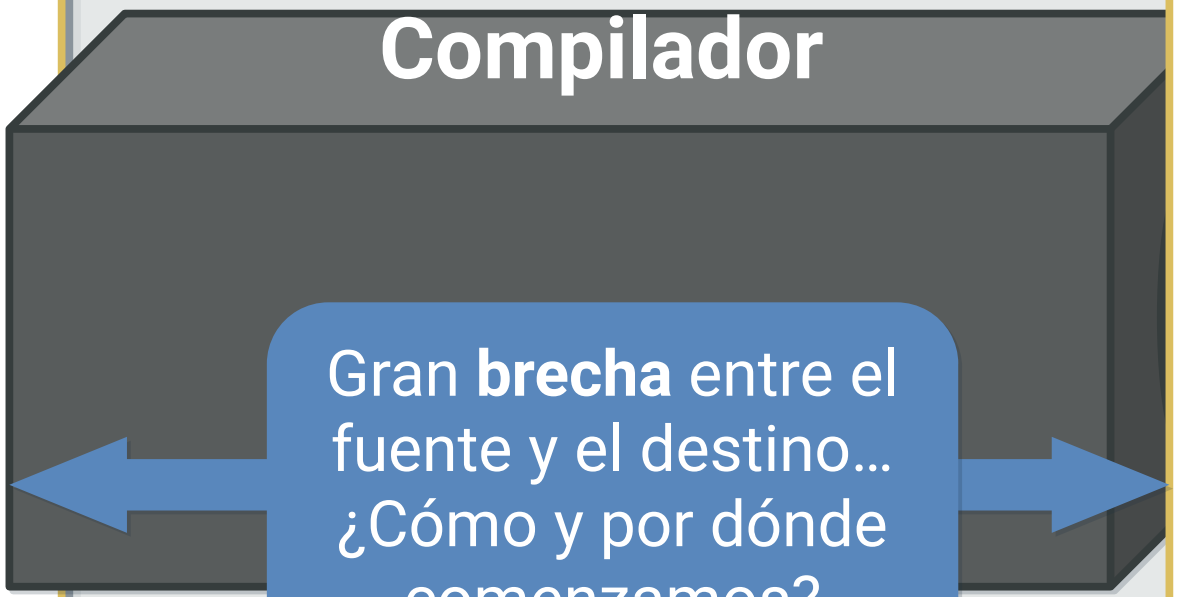
Características de un Compilador

- Un **Compilador** es un software que **traduce** automáticamente un programa de un lenguaje **fuente** a un ivalente de un lenguaje **destin**

Lenguaje Alto Nivel

- Independiente de la máquina
- Sintaxis compleja
- Semántica compleja
- Anidamiento
- Subprogramas
- Objetos

…



Gran **brecha** entre el fuente y el destino… ¿Cómo y por dónde comenzamos?

Lenguaje Bajo Nivel

- Específico
- Sintaxis simple
- Semántica simple
- Manejo de memoria
- Registros
- Alcance simple

…

Características de un Compilador

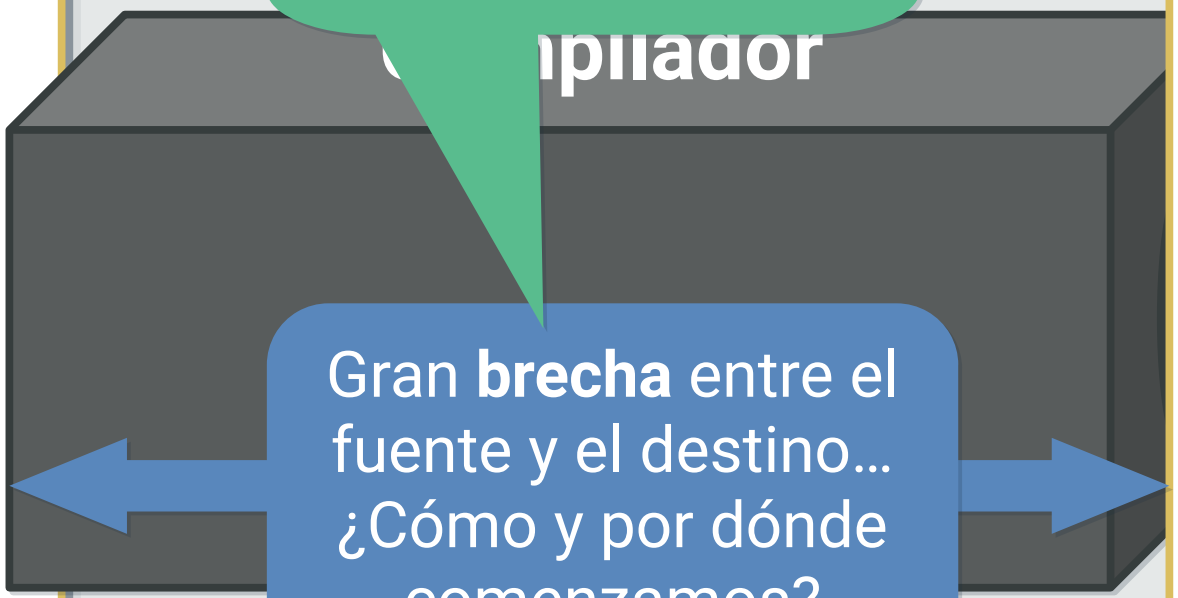
- Un **Compilador** es un software que **traduce** automáticamente un lenguaje **fuente** a un equivalente lenguaje **destino**

Lenguaje Alto Nivel

- Independiente de la máquina
- Sintaxis compleja
- Semántica compleja
- Anidamiento
- Subprogramas
- Objetos

...

Dividir y Conquistar!



Gran **brecha** entre el fuente y el destino... ¿Cómo y por dónde comenzamos?

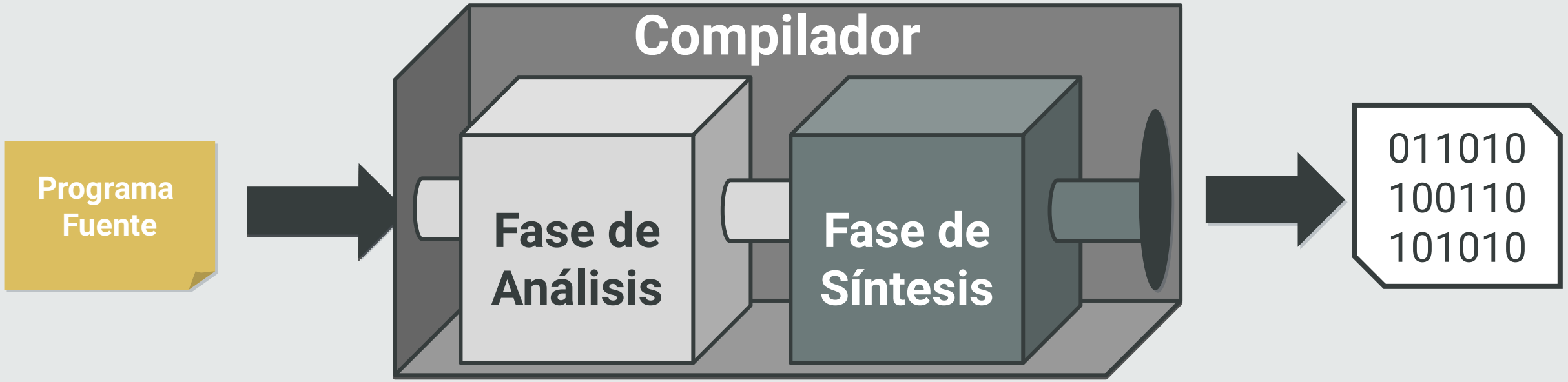
Lenguaje Bajo Nivel

- Específico
- Sintaxis simple
- Semántica simple
- Manejo de memoria
- Registros
- Alcance simple

...

Características de un Compilador

- Las tareas del compilador se **dividen en fases**, las cuales se **agrupan** de la siguiente manera:

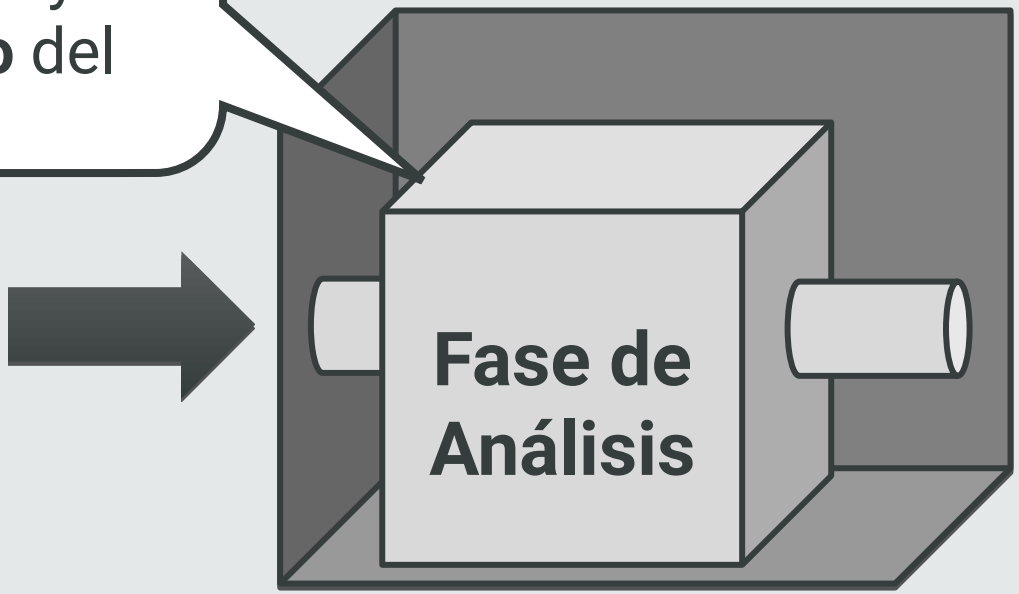


Fase de Análisis

Fase de Análisis

Analizar y recopilar la información de la **estructura** y el **significado** del fuente

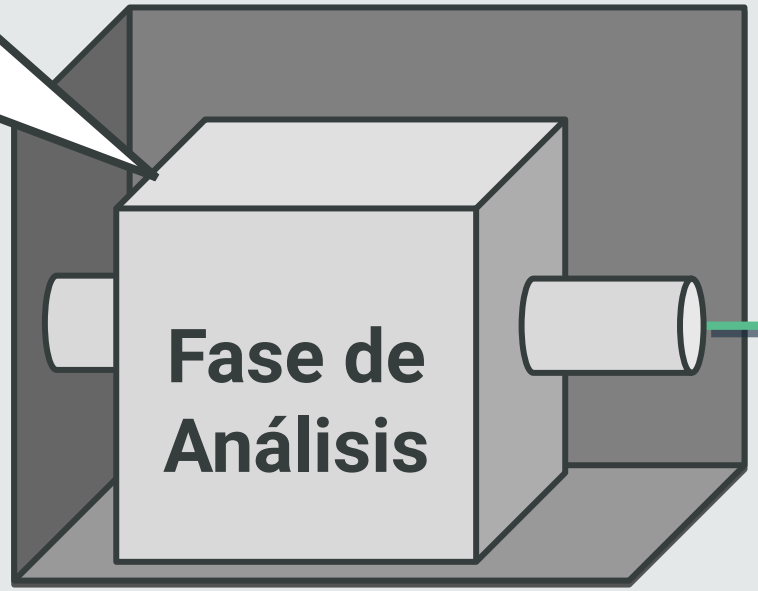
Programa Fuente

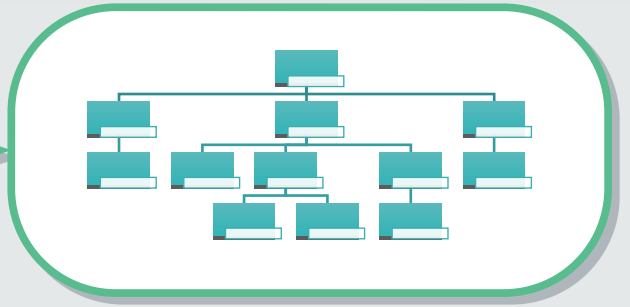


Fase de Análisis

Analizar y recopilar la información de la **estructura** y el **significado** del fuente

Programa Fuente

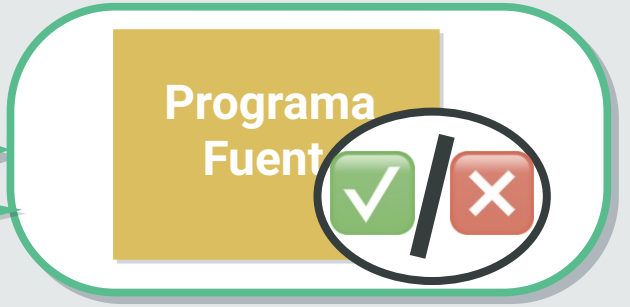




Nombre	Retorno	Params
m1	void	-

Nombre	tipo
x	float
y	float

...

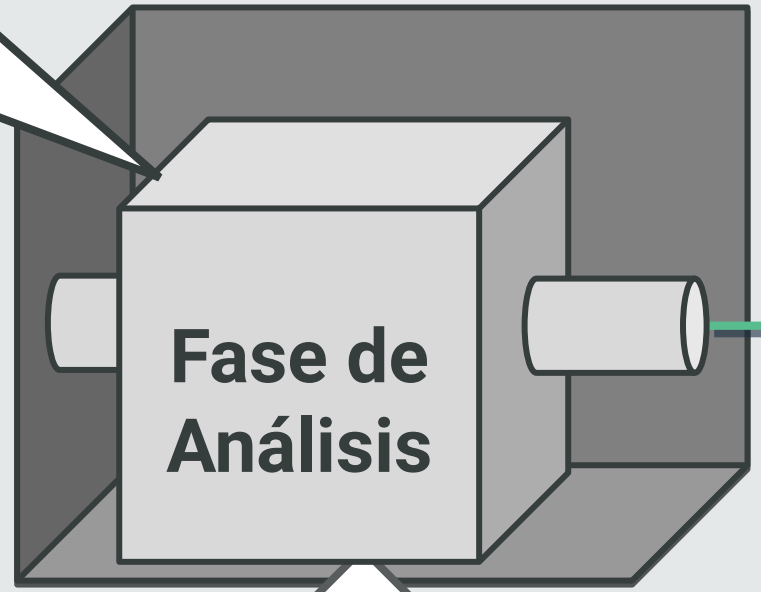


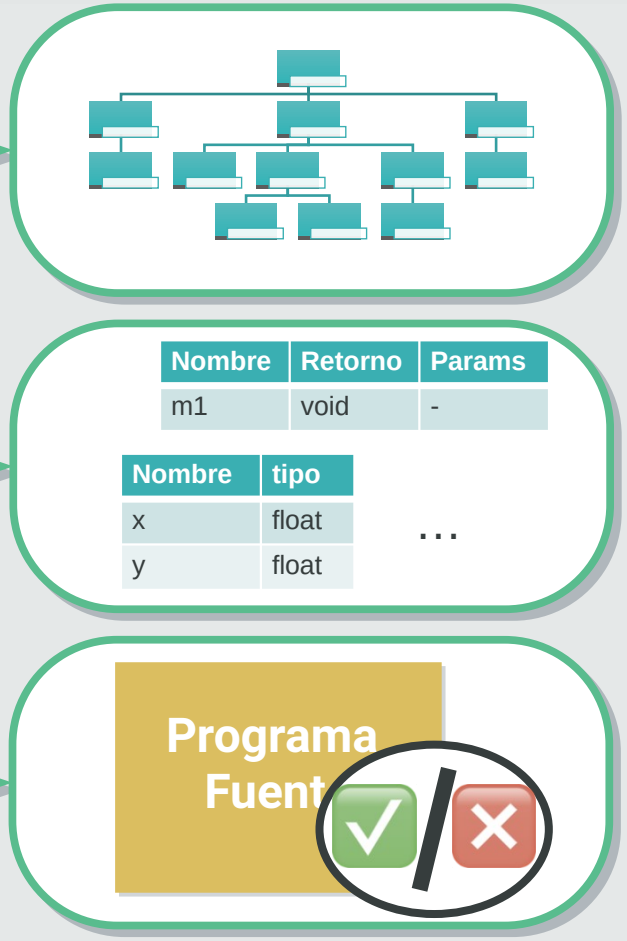
Verificar la correctitud sintáctica y semántica del programa; en caso de no ser correcto, **reportar el error** con la información adecuada.

Fase de Análisis

Analizar y recopilar la información de la **estructura** y el **significado** del fuente

Programa Fuente



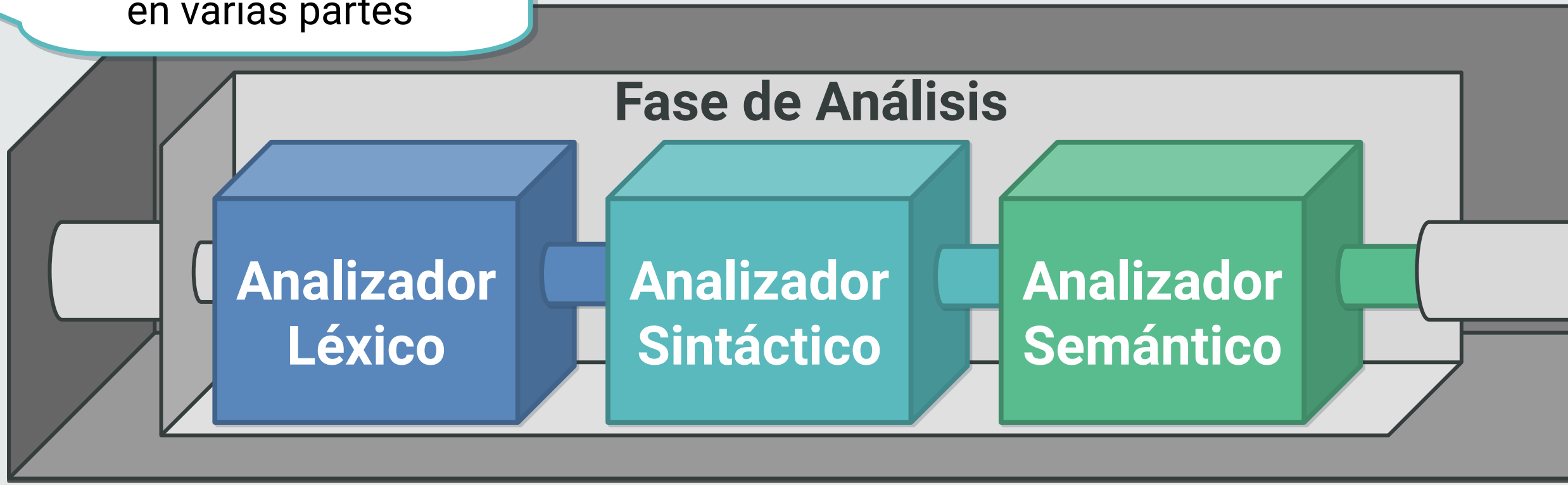


Para realizar estas tareas el compilador debe valerse de la especificación del lenguaje fuente - de sus reglas de Sintaxis y Semántica

Especificación del Lenguaje fuente

Fase de Análisis

La fase de análisis también está subdivida en varias partes



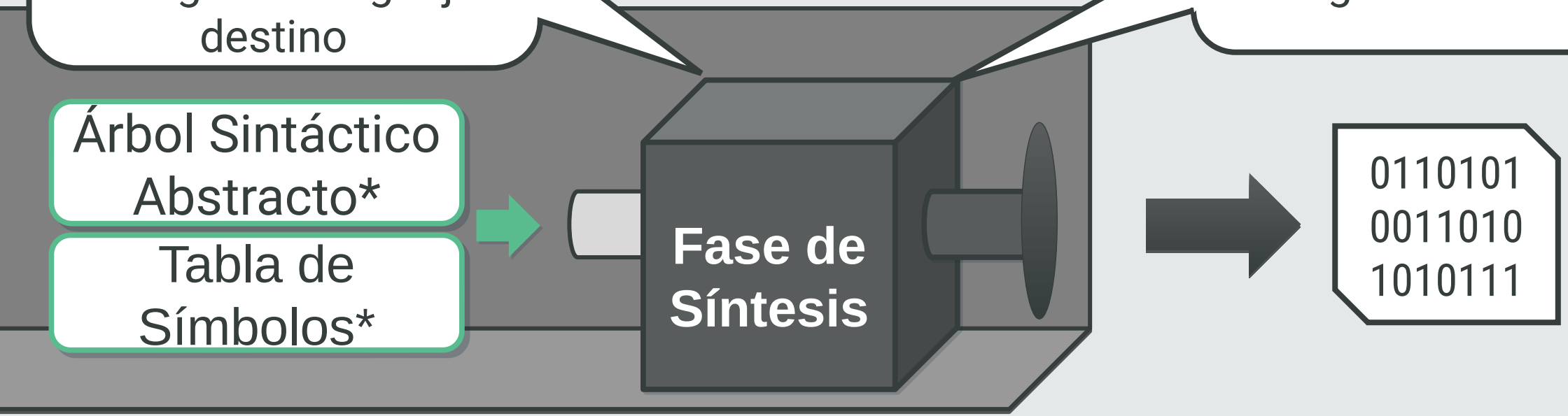
¡Dividir y Conquistar!

Fase de Síntesis

Fase de Síntesis

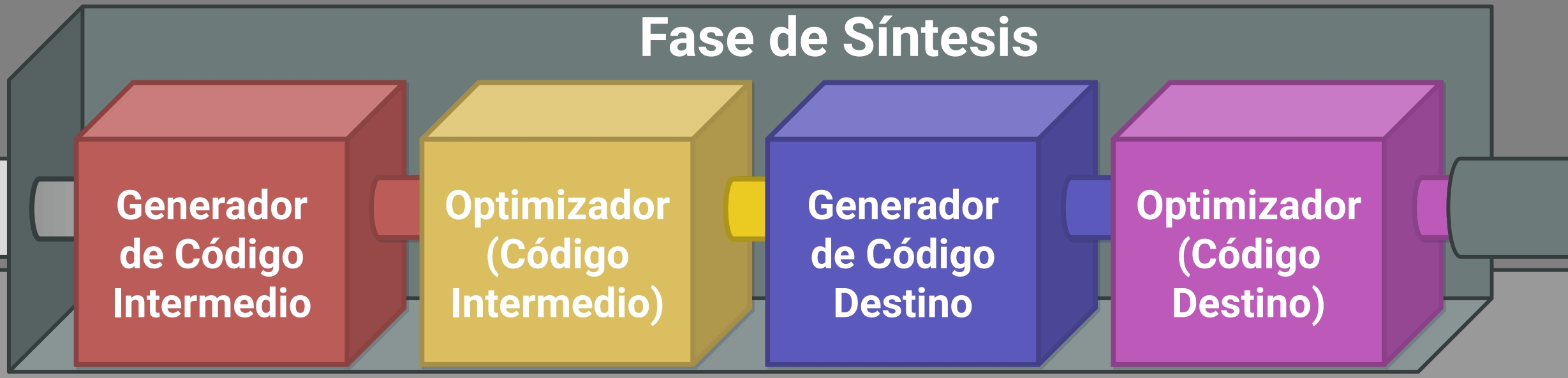
Toma la información recopilada por la fase de análisis y traduce a código del lenguaje destino

Busca que el resultado sea un código eficiente



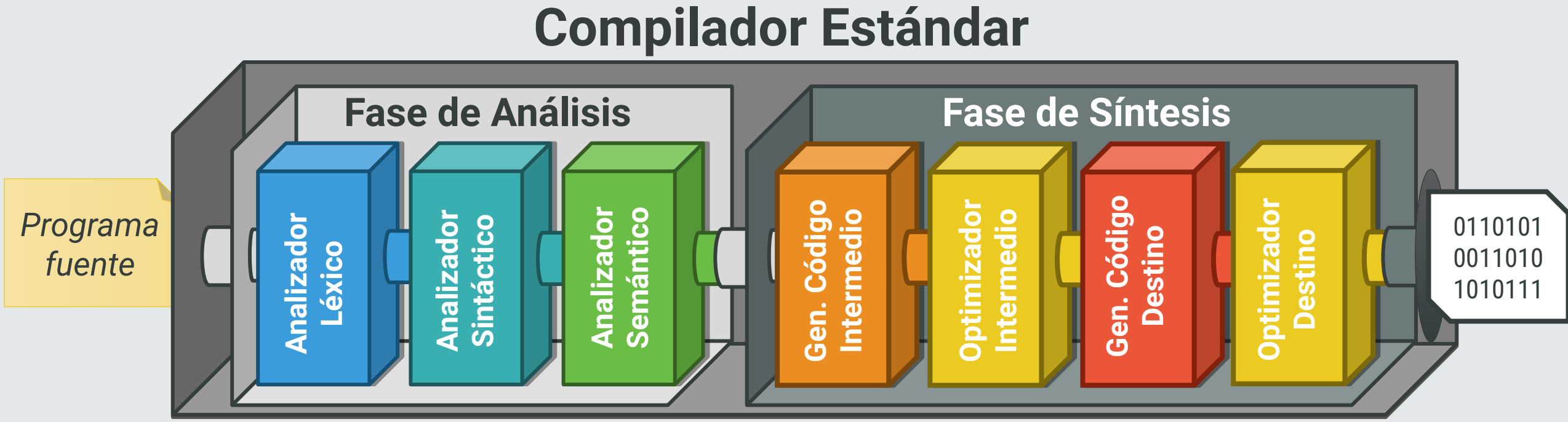
Fase de Síntesis

La fase de síntesis también esta subdivida en varias partes

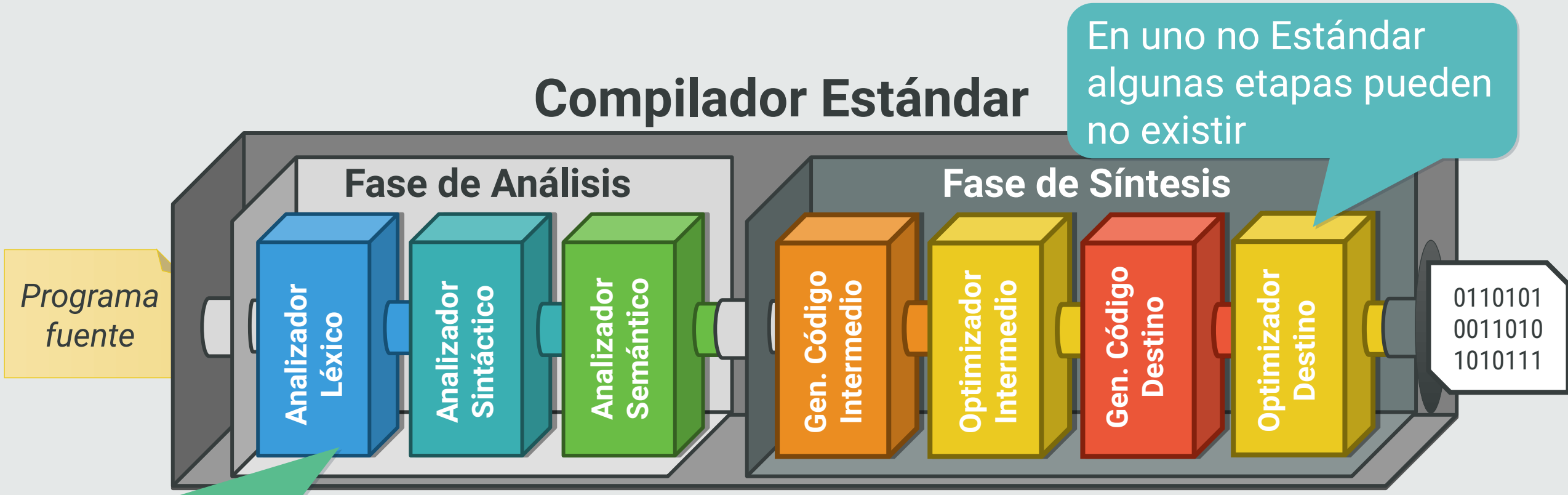


¡Dividir y Conquistar!

Estructura de un Compilador



Estructura de un Compilador

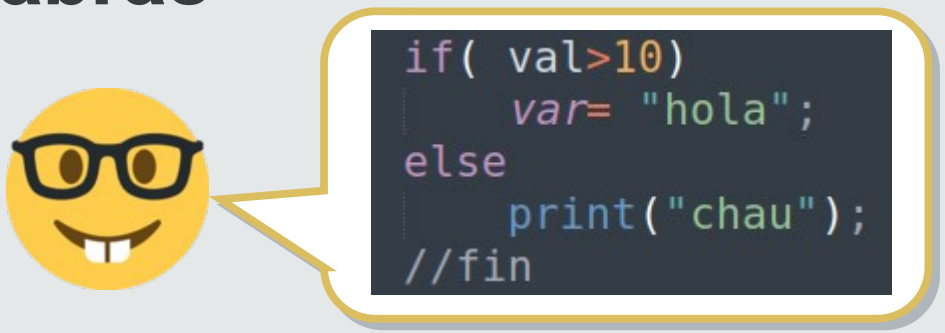


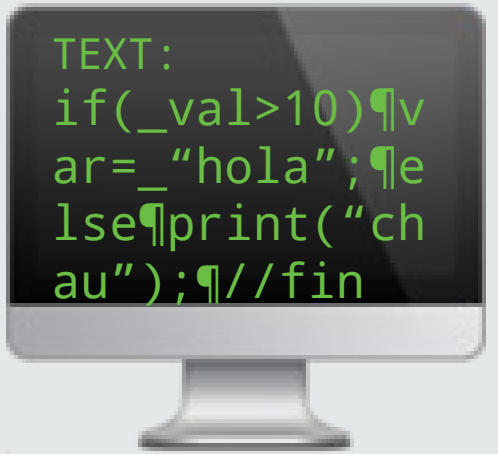
Compiladores e Intérpretes

Análisis Léxico – Esquema General

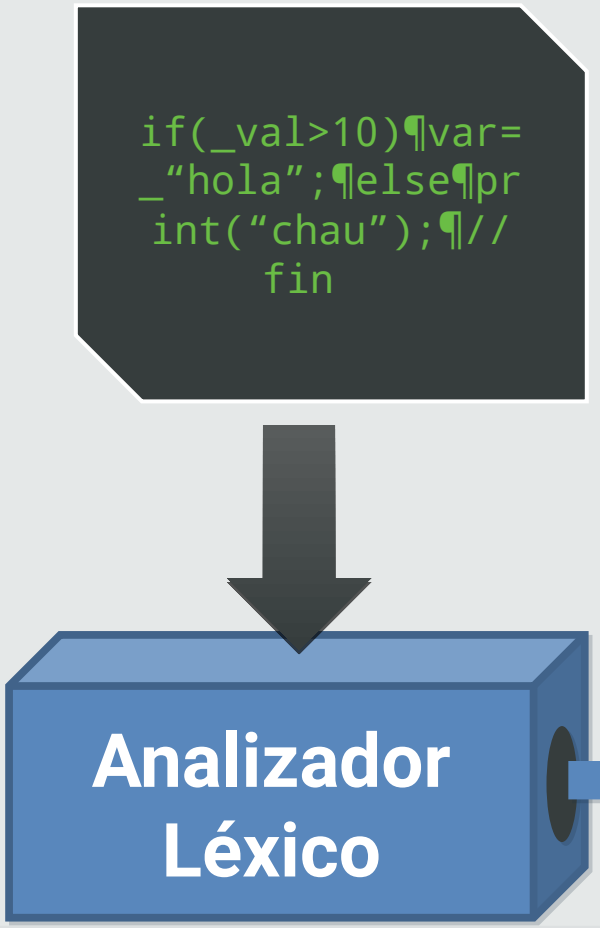
Esquema General del Análisis Léxico

- El **programa fuente** es una cadena de caracteres
- Tenemos que reconocer si esa cadena **pertenece al lenguaje** utilizando las reglas de **sintaxis del lenguaje**
- Las reglas de sintaxis se expresan en términos de “**palabras**”





Esquema General del Análisis Léxico

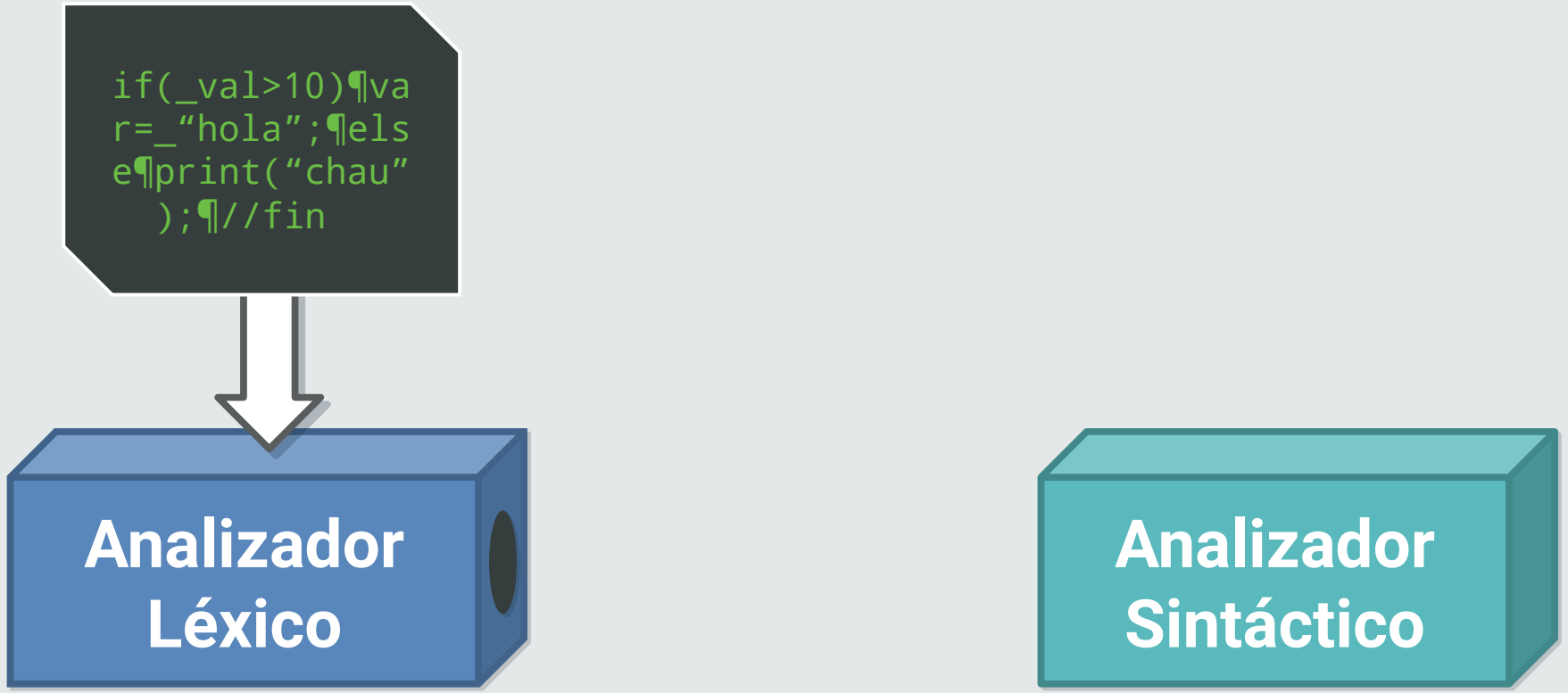


- El **Analizador Léxico** es el encargado de leer “directamente” el **texto** del **programa fuente**
- Busca **construir** esas **palabras** o subcadenas **agrupando caracteres**
- **Elimina** blancos, separadores y comentarios
- Informa de **Errores Léxicos**

if	(	id	>	num	)	id	=	String	;	else	print	(	String	)	;

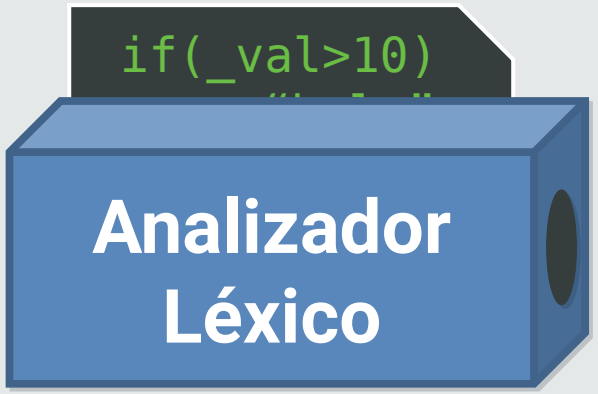
Esquema General del Análisis Léxico

- El **Analizador Léxico** lee los caracteres directo del fuente



Esquema General del Análisis Léxico

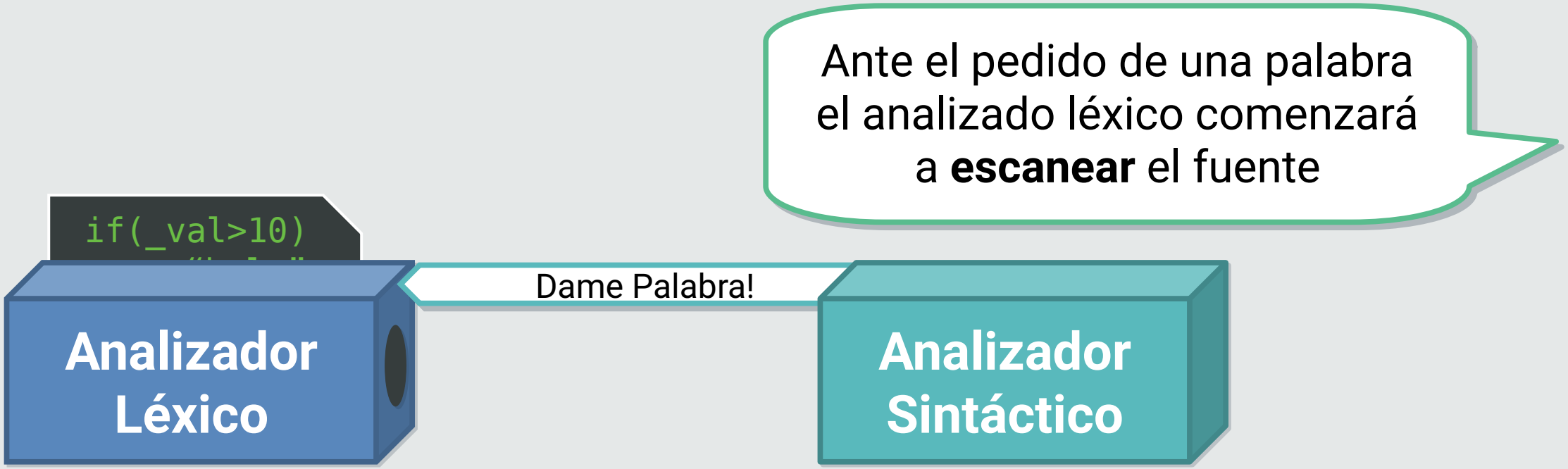
- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá solicitándole palabras





Esquema General del Análisis Léxico

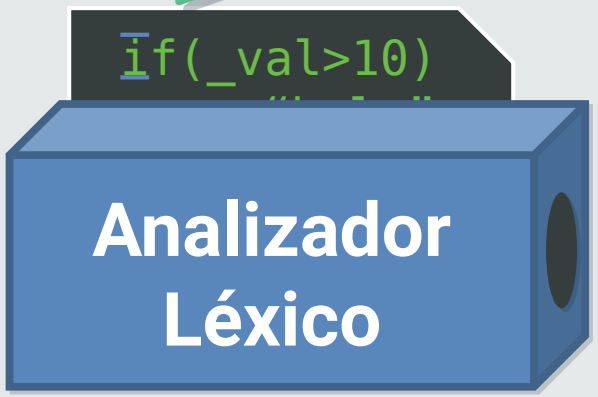
- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá solicitándole palabras



Esquema General del Análisis Léxico

- El **Analizad** anda!
- El analizad e palabras

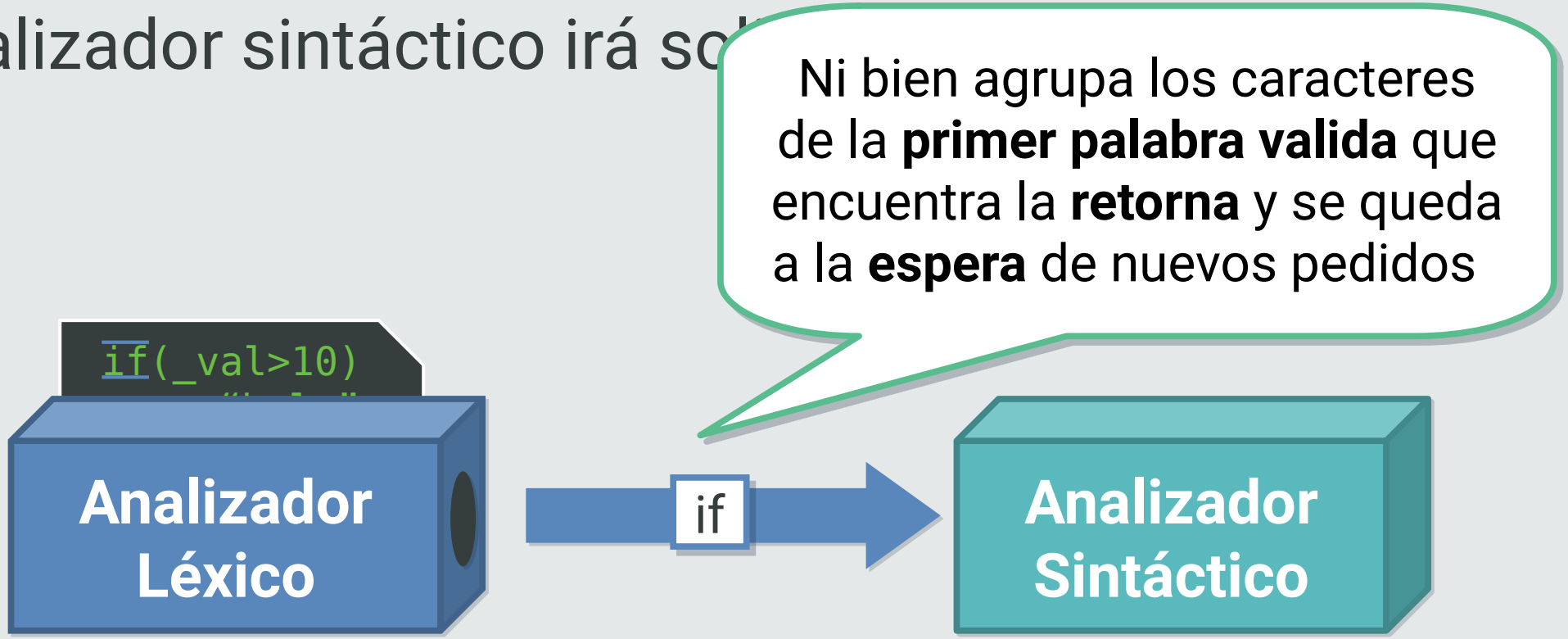
Para esto va recorriendo y **agrupando caracteres** hasta que los caracteres agrupados formen una **palabra valida**



Analizador Sintáctico

Esquema General del Análisis Léxico

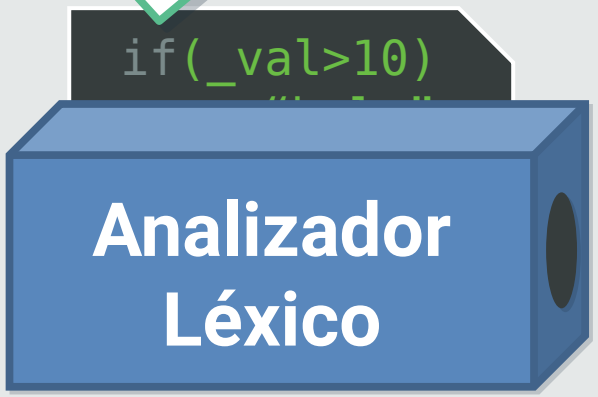
- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá so



Esquema General del Análisis Léxico

- E… …r demanda!
- E… …tándole palabras

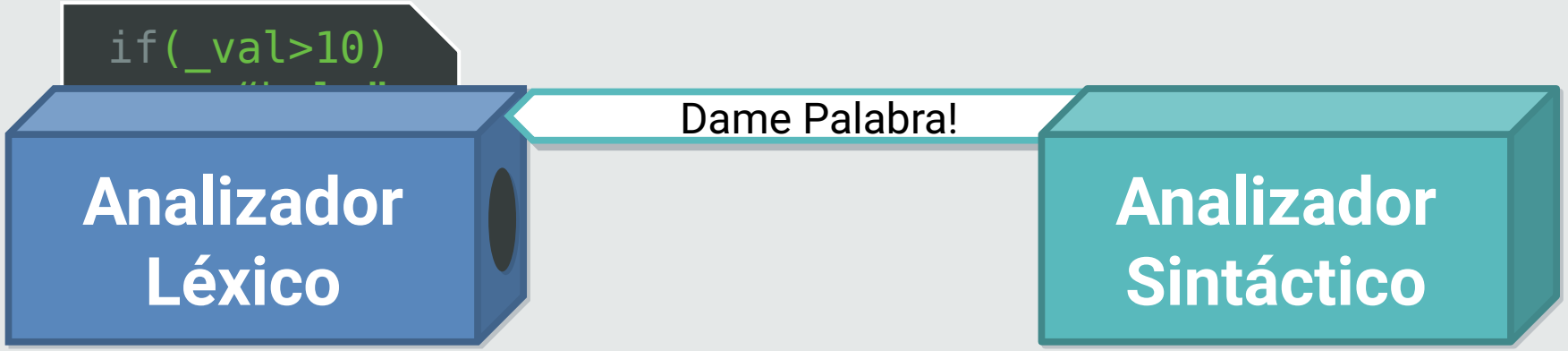
Los **caracteres** que **usó** para formar la palabra **son consumidos** y determinan desde **donde arrancará** a escanear ante un **nuevo pedido**





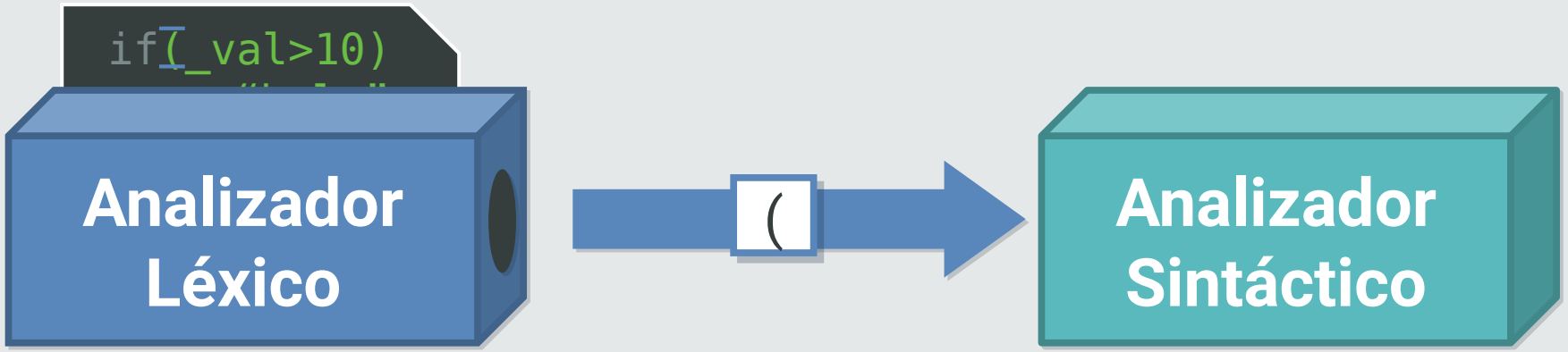
Esquema General del Análisis Léxico

- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá solicitándole palabras



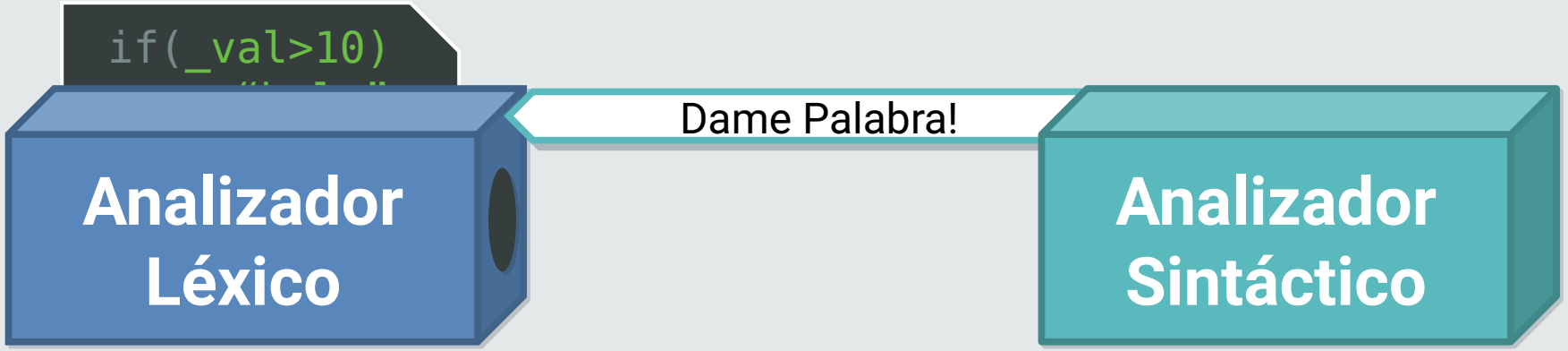
Esquema General del Análisis Léxico

- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá solicitándole palabras



Esquema General del Análisis Léxico

- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá solicitándole palabras

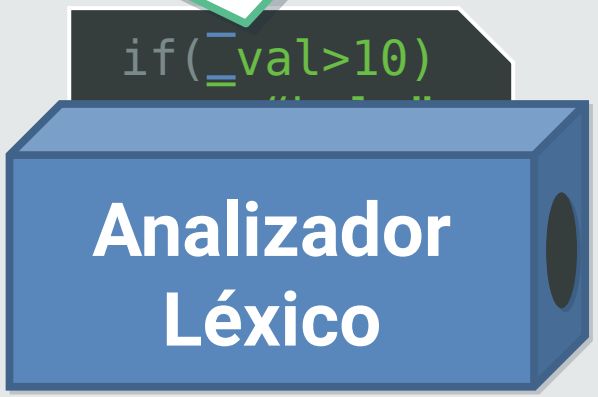


Esquema General del Análisis Léxico

- El A… …por demanda!
- El a… …citán

Cuando arranca un nuevo pedido y se encuentra con uno o mas **blancos** (espacio, tab, enter, etc) los **consume** y lo **descarta**

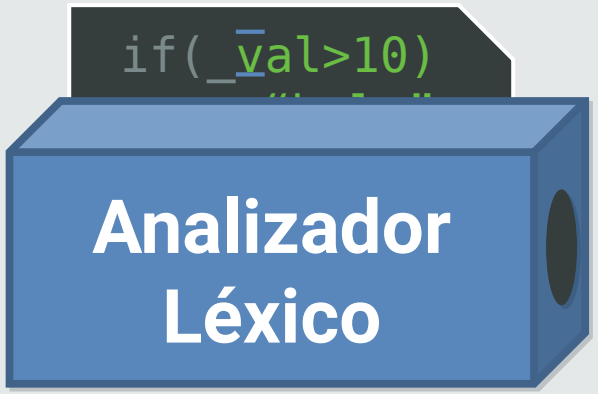
Es decir los ignora y no forman parte de la palabra valida



Analizador Sintáctico

Esquema General del Análisis Léxico

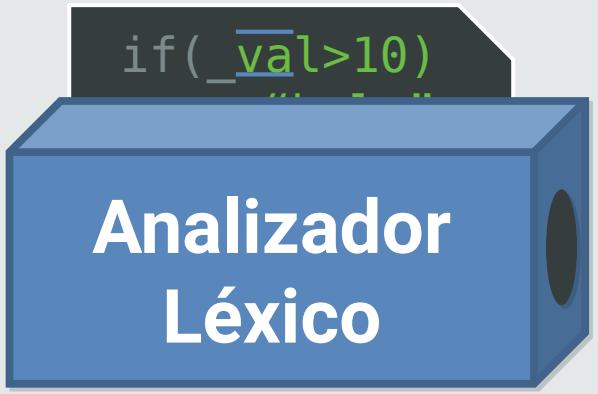
- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá solicitándole palabras





Esquema General del Análisis Léxico

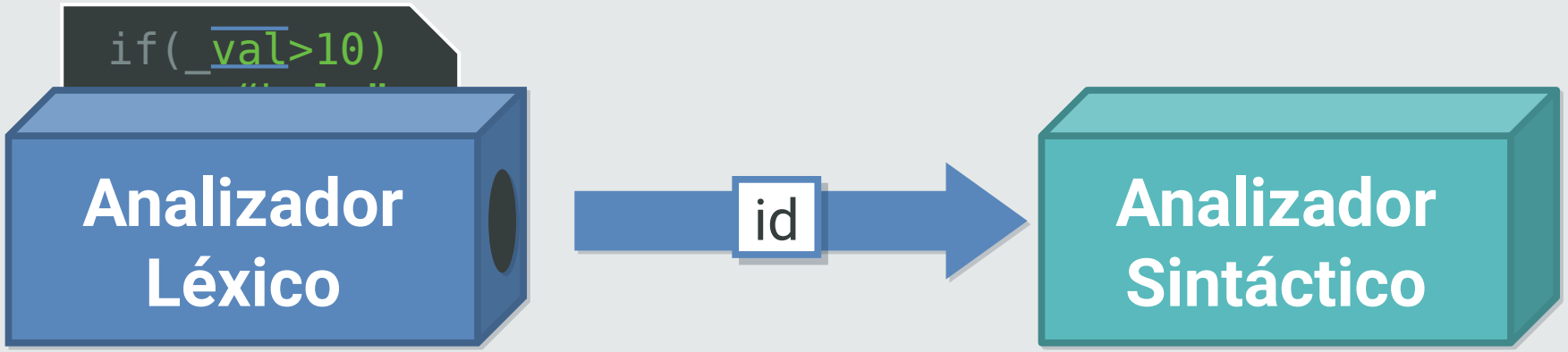
- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá solicitándole palabras





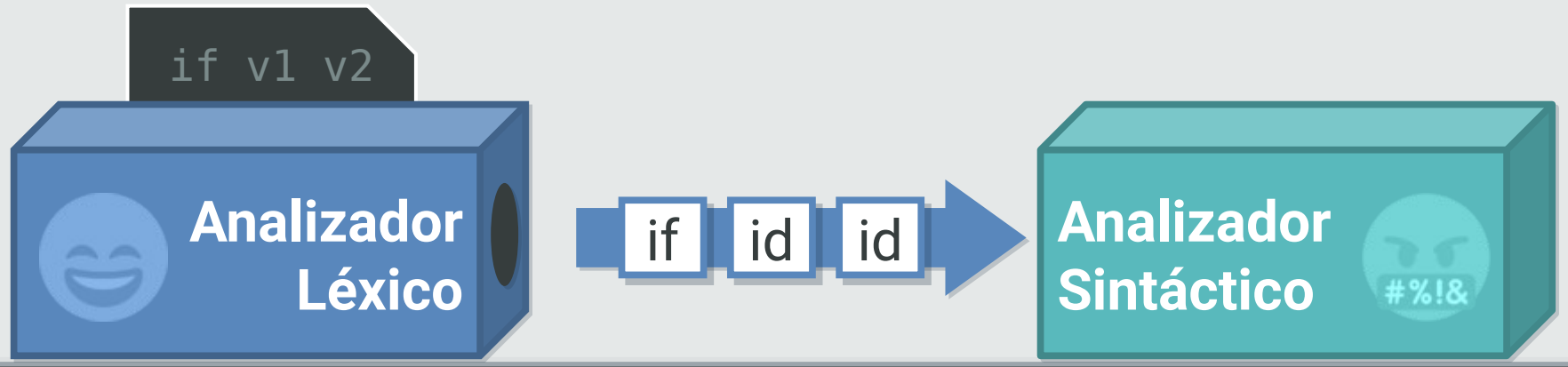
Esquema General del Análisis Léxico

- El **Analizador Léxico** funciona por demanda!
- El analizador sintáctico irá solicitándole palabras



Esquema General del Analizador Lexico

- Al **Analizador Léxico no le importa el orden** en que aparecen las palabras validas en el fuente!
  - **Su trabajo** es solamente **reconocerlas y detectar errores** cuando aparece algo que no es una palabra valida o un blanco
  - El que se encarga de ver que las **palabras validas** aparecen en **orden** correcto es el **Analizador Sintactico**



Especificando Tokens – Expresiones Regulares

Especificando Tokens – Expresiones Regulares

- Para **implementar** el **analizador léxico** de un lenguaje de programación tenemos que **identificar** sus **tokens**
- Para esto, para cada token, tenemos que determinar el **patrón** que caracteriza a sus **lexemas**, es decir:

Dar una **especificación para los tokens** del lenguaje

Especificando Tokens – Expresiones Regulares

- El formalismo usual para esta tarea son los **Lenguajes Regulares**
  - ¿Por qué estos lenguajes?

Especificando Tokens – Expresiones Regulares

- El formalismo usual para esta tarea son los **Lenguajes Regulares**
  - ¿Por qué estos lenguajes?

Son lenguajes formales más **simples** y **eficientes** con el **poder expresivo** suficiente para identificar los lexemas que caracterizan a un token.

Especificando Tokens – Expresiones Regulares

- ¿Cómo es la expresión regular para un entero?
- ¿Como sería la expresión regular para un identificador?
- ¿Como serían para mayor y para mayor e igual?

Por simplicidad usamos [0..9], [a..z] y [A..Z] para caracterizar dígitos, letras minúsculas y letras mayúsculas respectivamente.

Especificando Tokens – Expresiones Regulares

- ¿Cómo es la expresión regular para un entero?

  Entero = `[0..9]+`

- ¿Como sería la expresión regular para un identificador?

  Identificador = `([a..z]|[A..Z])([a..z]|[A..Z]|[0..9])*`

- ¿Como serían para mayor y para mayor o igual?

  Mayor = `>`
  MayorIgual = `>=`

Por simplicidad usamos [0..9], [a..z] y [A..Z] para caracterizar dígitos, letras minúsculas y letras mayúsculas respectivamente.

Especificando Tokens – Expresiones Regulares

- ¿Cómo es la expresión regular para un entero?

  Entero = `Digito+`

- ¿Como sería la expresión regular para un identificador?

  Identificador = `Letra(Letra|Digito)*`

- ¿Como serían para mayor y para mayor o

  Mayor = `>`
  MayorIgual = `>=`

Digito = `[0..9]`
Letra = `[a..z]|[A..Z]`

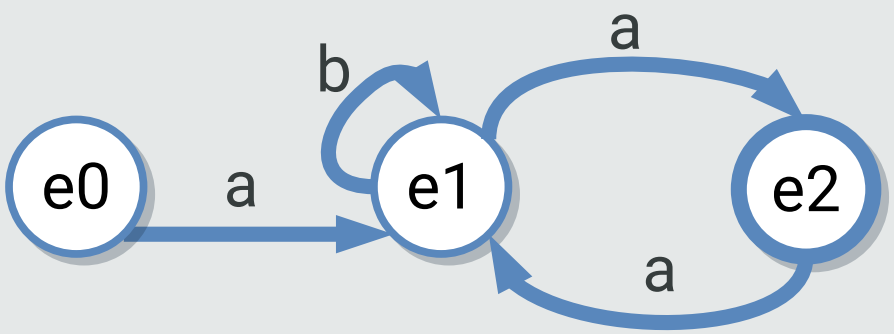
Otra forma mas conveniente de notarlos usando las ER para Dígito y Letra

Por simplicidad usamos [0..9], [a..z] y [A..Z] para caracterizar dígitos, letras minúsculas y letras mayúsculas respectivamente.

Reconociendo Tokens – Autómatas Finitos

Reconociendo Tokens – Autómatas Finitos

- El AF va **aplicando transiciones** a medida que **consume** los **caracteres** de la cadena de entrada
- Partiendo del estado inicial, si el autómata puede consumir toda la toda la cadena y queda en un estado finalizador **acepta la cadena** sino **rechaza la cadena**



abba ✓

aabb ✗

abbaa ✗

Reconociendo Tokens – Autómatas Finitos

- Dada una **ER** podemos construir un **AF** que reconoce el lenguaje generado por esa expresión
- ¿Cómo sería el autómata para reconocer X? donde X es:

Operador Mayor

Operador Mayor o Igual

Enteros

Identificadores

Reconociendo Tokens – Autómatas Finitos

- Dada una **ER** podemos construir un **AF** que reconoce el lenguaje generado por esa expresión
- ¿Cómo sería el autómata para reconocer X? donde X es:

